ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Môn: Mạng máy tính (CO3039)

ASSIGNMENT 1: DISTRIBUTED AND STREAMING-BASED CLIENT-SERVER COLLABORATION APPLICATION

Giảng viên hướng dẫn : Bùi Xuân Giang
Lớp : L09
Nhóm : 5

STT	Sinh viên thực hiện	MSSV
1	Nguyễn Thanh Nguyên	2412359
2	Bùi Quang Đức	2410796
3	Đỗ Anh Khoa	2452526
4	Phạm Quốc Huy	2411261
5	Nguyễn Thị Thúy Hằng	2310904

Tp. Hồ Chí Minh Ngày 14 tháng 11 năm 2026

BẢNG PHÂN CÔNG NHIỆM VỤ

Nhóm: **5** Lớp: **L09** Môn: **Mạng máy tính** Giảng viên: **Bùi Xuân Giang**

STT	Họ và tên	MSSV	Nhiệm vụ	Ký tên
1	Nguyễn Thanh Nguyên	2412359	Thiết kế giao thức ứng dụng; Xây dựng UDP relay theo phiên; Truyền file tin cập qua UDP	
2	Bùi Quang Đức	2410796	Kết nối TCP/TLS, handshake và xác thực; Quản lý phiên, xử lý đồng thời nhiều client và định tuyến tin nhắn	
3	Phạm Quốc Huy	2411261	Bảo mật: TLS, cấp khóa theo phiên, chống relay; Giao diện Web; Kiểm thử hệ thống	

Mục lục

1 Giới thiệu 1

1.1 Mục tiêu 1

1.2 Phạm vi và công nghệ 1

2 Kiến trúc tổng quan 2

3 Thiết kế giao thức 4

3.1 Lực chọn thiết kế 4

3.2 Giao thúc điều kiển TCP 4

3.3 Giao thức file trên UDP 6

3.4 Máy trạng thái 7

4 Server core 9

4.1 Kết nối và handshake 9

4.2 Xác thực và quản lý phiên 9

4.3 Xử lý đồng thời (Concurrency) 9

4.4 Định tuyến tin nhắn 10

4.5 Xử lý lỗi và độ bền (Resillience) 11

5 Truyền file UDP tin cậy 12

5.1 Chia chunk và tái lắp ghép 12

5.2 Gửi lại có chọn lọc (selective repeat) 12

5.3 Kiểm soát cửa số và tốc độ 12

5.4 Thông điệp điều khiển cũng có thể mất 13

5.5 Xác minh và công bố file 13

5.6 Hủy, hết hạn và dọn dẹp 13

5.7 Cô lập giữa các đường truyền và giới hạn tài nguyên 13

5.8 Kết quả kiểm chứng và giới hạn 14

6 Bảo mật 15

6.1 Mô hình tin cậy và phạm vi 15

6.2 TLS và HTTPS 15

6.3 Mã hóa payload bằng AES-256-GCM 15
6.4 Quản lý khóa theo phiên 16
6.5 Nonce, dữ liệu xác thực kèm theo (AAD) và chống replay 16
6.6 Xử lý lỗi bảo mật và tính đúng đắn của kết quả 17
6.7 Phạm vi kiểm chứng và giới hạn 17
7 Giao diện Web 18
7.1 Kiến trúc: trình duyệt, bridge và backend 18
7.2 Luồng sử dụng 18
7.3 Khả nawg truy cập và thích ứng thiết bị 20
7.4 Giới hạn 20
8 Kiểm thử hệ thống 21
8.1 Công cụ và quy trình 21
8.2 Các nhóm kiểm thử 21
8.3 Kết quả 22
8.4 Hạn chế của kiểm thử 22
9 Hạn chế và phát triển 23
10 Kết luận 26

1. Giới thiệu

1.1. Mục tiêu

Assignment 1 yêu cầu xây dựng một ứng dụng cộng tác theo mô hình client-server, cho phép nhiều người dùng đăng nhập, nhắn tin chung và riêng, truyền file và theo dõi người đang hoạt động qua giao diện Web. Nhóm đặt tên ứng dụng là **Connect**. Hệ thống gồm một server xử lý xác thực và định tuyến tin nhắn qua TCP/TLS, một UDP relay truyền file tin cậy, một lớp bảo mật mã hóa dữ liệu bằng AES-256-GCM, và một giao diện Web dùng HTML, CSS và JavaScript phía client.

1.2. Phạm vi và công nghệ

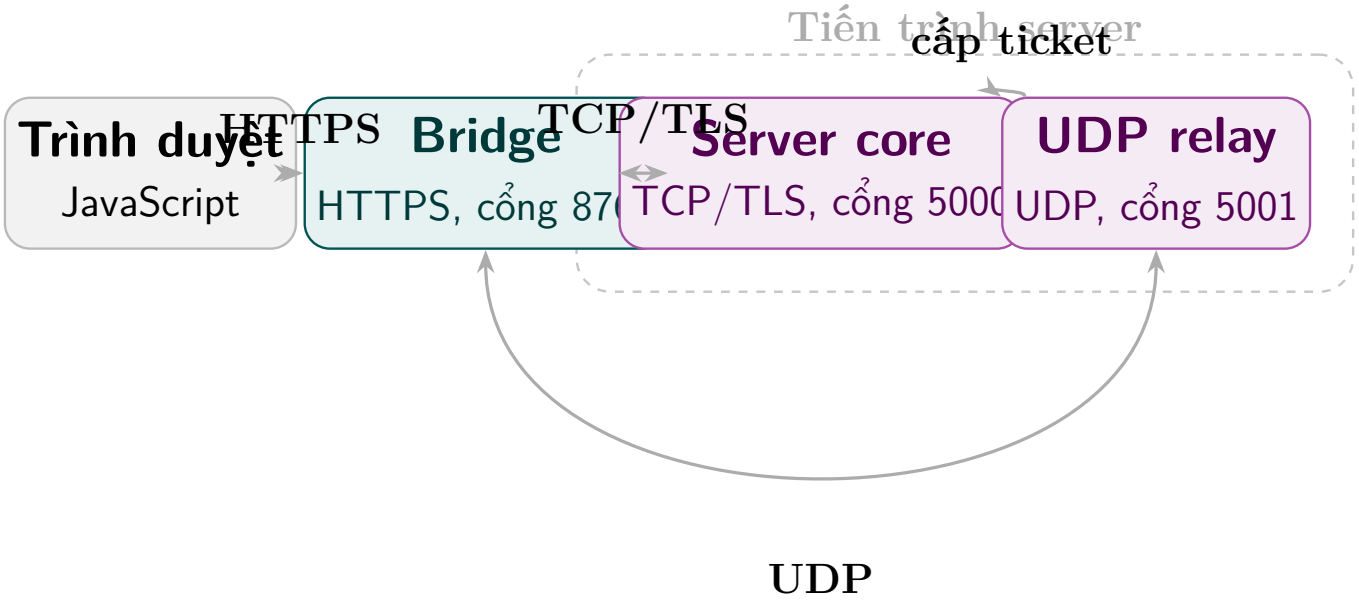
Hệ thống chạy trên Windows với Python 3.14, dùng socket của thư viện chuẩn cho TCP và UDP, module `ssl` cho TLS và thư viện `cryptography` cho AES-GCM. Theo ràng buộc của đề, nhóm không dùng Flask/Django, WebSocket hay thư viện mạng cấp cao, và không thêm framework mạng nào ngoài các thư viện kể trên. Phạm vi kiểm chứng là localhost; hệ thống chưa được kiểm thử trên mạng LAN nhiều máy.

2. Kiến trúc tổng quan

Hệ thống gồm bốn thành phần chạy trong hai tiến trình chính, cộng một giao diện chạy trong trình duyệt:

Thành phần	Thư mục	Nhiệm vụ
Server core	`server/`	Nhận kết nối TCP/TLS, xác thực, quản lý phiên, định tuyến chat, cấp phiên file
UDP relay và module truyền file	`p1/`, `shared/`	Chuyển tiếp gói UDP theo phiên, truyền file tin cậy; relay chạy cùng tiến trình với server
Bridge	`bridge.py`, `bridge_files.py`	Phục vụ trang Web qua HTTPS, nối trình duyệt với server và relay, giữ khóa phiên
Giao diện Web	`web/`	Hiển thị chat, người dùng online, thẻ file và tiến trình
Bảo mật	`security/`	Sinh chứng thư thử, TLS, AES-GCM, quản lý nonce và chống replay

Hình 1: Kiến trúc tổng quan của hệ thống



Trình duyệt gửi yêu cầu tới bridge qua HTTPS. Bridge duy trì kết nối TCP/TLS tới server cho từng phiên đăng nhập, server xác thực, giải mã tin, rời định tuyến và mã hóa lại cho người nhận. Tin đến người nhận đi ngược lại qua bridge của họ tới trình duyệt.

Bridge nhận file từ trình duyệt và giao cho một worker dùng `FileClient` gửi qua UDP relay. Trước đó, server core cấp ticket cho phiên (yêu cầu `FILE_SESSION`) và thu hồi ticket khi người dùng

ngắt kết nối. Relay chuyển tiếp gói UDP đã mã hóa tới worker của người nhận, không lưu nội dung file.

Trình duyệt không bao giờ giữ khóa phiên hay ticket, chúng chỉ nằm ở bridge. Kiến trúc tách riêng xử lý mạng, xử lý dữ liệu và giao diện, đúng yêu cầu của đề. Việc truyền file qua relay thay vì giữa hai client với nhau được giải thích ở mục 3.1.

3. Thiết kế giao thức

3.1. Lực chọn thiết kế

Ứng dụng dùng 2 kênh có vai trò khác nhau:

- TCP/TLS cho xác thực, quản lý phiên và chat. Luồng byte có thứ tự và tin cậy của TCP phù hợp với các thông điệp điều khiển ngắn.
- UDP cho truyền file. Độ tin cậy (xác nhận, gửi lại, kiểm tra toàn vẹn) được cài đặt ở tầng ứng dụng.

File không đi trực tiếp giữa hai client mà đi qua một UDP relay theo phiên. Relay chuyển tiếp dữ liệu nhưng không lưu nội dung file. Nhờ vậy client không cần tự dò tìm địa chỉ của người nhận, và relay có thể kiểm tra từng gói theo phiên đã xác thực.

Server TCP chỉ ghi nhận cổng TCP của client, mà cổng này không cho biết socket UDP nhận file nằm ở đâu. Vì vậy mỗi client phải đăng ký điểm cuối UDP của mình với relay bằng một thủ tục có thử thách (challenge), mô tả ở mục 3.3.

Toàn bộ `OFFER`, `ACCEPT`, `DATA`, `ACK`, `FIN` và `COMPLETE` của file đi qua UDP. Chúng không phải là action TCP và không đi qua `MessageRouter` của server.

3.2. Giao thúc điều kiển TCP

Mặc định server dùng cổng TCP 5000 và relay dùng cổng UDP 5001, cả hai đều cấu hình được. Sau handshake `HELLO2`/`WELCOME2` trong TLS, mỗi thông điệp là một đối tượng JSON UTF-8 kết thúc bằng LF; ký tự xuống dòng bên trong nội dung phải được escape. Bên nhận phải đệm dữ liệu và tách từng dòng hoàn chỉnh, vì một lần recv có thể chứa nửa thông điệp hoặc nhiều thông điệp.

Ba loại thông điệp được phân biệt bằng trường định danh:

Loại	**Trường**	**Hướng**
Yêu cầu	`action`	Client → Server
Phản hồi	`status`	Server → Client
Sự kiện chat bất đồng bộ	`type`	Server → Client

Mã trạng thái là chuỗi chữ hoa, phân biệt hoa thường, không phải mã HTTP. Danh sách action và status đầy đủ nằm ở mục 4.4. Ví dụ một phiên trao đổi (dạng logic, trước lớp mã hóa chat ở mục 6):

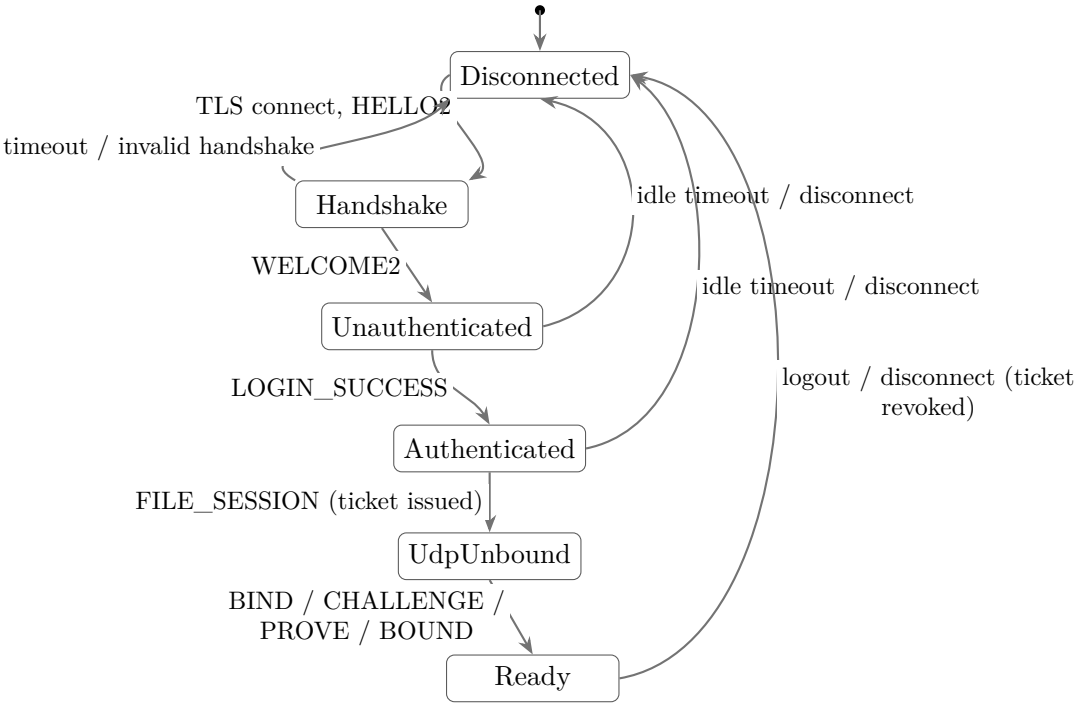
```
{"action":"REGISTER","username":"alice","password":"example-only"}
{"status":"REGISTER_SUCCESS"}

{"action":"LOGIN","username":"alice","password":"example-only"}
{"status":"LOGIN_SUCCESS"}

{"action":"UNICAST","recipient":"bob","message":"Hello"}
{"type":"UNICAST","sender":"alice","recipient":"bob","message":"Hello"}
{"status":"UNICAST_SUCCESS"}

{"action":"LIST_ONLINE"}
{"status":"LIST_ONLINE_SUCCESS","users":["alice","bob"]}
```

Dòng thứ sáu là sự kiện server gửi cho bob, các dòng còn lại là phản hồi cho alice. Do giao thức không có mã yêu cầu, một kết nối chỉ có một yêu cầu chờ phản hồi tại một thời điểm, và bên đọc phải tách `status` khỏi `type` vì sự kiện có thể xen giữa các phản hồi.



Hình 2: State machine của kết nối client

3.3. Giao thức file trên UDP

Mỗi datagram là một gói hoàn chỉnh, các số nguyên theo thứ tự byte mạng (network byte order). Phần đầu (header) dài 44 byte:

Offset	Byte	Trường
0	4	Magic `P1FT`
4	1	Phiên bản giao thức (hiện là 2)
5	1	Loại thông điệp
6	16	UUID phiên của client trên chặng relay này
22	16	UUID của lượt truyền; toàn số 0 với thông điệp cấp phiên
38	4	Số thứ tự chunk bắt đầu từ 0; bằng 0 với thông điệp điều khiển
42	2	Độ dài payload đã mã hóa

Sau header là phần thân: `nonce` 12 byte, dữ liệu mã hóa (0 đến 996 byte), thẻ GCM 16 byte và thẻ HMAC 16 byte. Mỗi chunk file mang tối đa 996 byte dữ liệu gốc, nên một datagram DATA dài tối đa 1.084 byte, còn gói điều khiển rỗng chỉ 88 byte. Kích thước này được chọn thận trọng để giảm nguy cơ phân mảnh IP trên mạng LAN thông thường; nó không thay thế việc dò MTU của đường truyền. Ý nghĩa của nonce, thẻ GCM và HMAC được giải thích ở mục 6.

Các loại thông điệp:

Giá trị	Loại	Ý nghĩa
1	`BIND`	Yêu cầu đăng ký điểm cuối UDP
2	`BOUND`	Xác nhận đã đăng ký điểm cuối
3	`OFFER`	Đề nghị gửi file, kèm siêu dữ liệu (metadata) JSON
4	`ACCEPT`	Người nhận đồng ý và đã cấp phát trạng thái nhận
5	`REJECT`	Người nhận từ chối, kèm lý do
6	`DATA`	Một chunk dữ liệu file
7	`ACK`	Người nhận đã ghi chunk đó
8	`FIN`	Người gửi đã nhận ACK cho mọi chunk
9	`COMPLETE`	Người nhận đã kiểm tra kích thước, SHA-256 và lưu file
10	`ERROR`	Mã lỗi
11	`CANCEL`	Hủy lượt truyền
12	`PING`	Giữ phiên UDP
13	`PONG`	Phản hồi PING
14	`CHALLENGE`	Số ngẫu nhiên 16 byte do relay cấp cho điểm cuối mới
15	`PROVE`	Gửi lại số ngẫu nhiên đó, kèm thẻ xác thực

Siêu dữ liệu OFFER gồm: `recipient` (tên người nhận), filename (tên tệp 1 đến 128 byte UTF-8, không chứa đường dẫn hay ký tự điều khiển), `size` (0 đến 268.435.456 byte, tức tối đa 256 MiB), `sha256` (64 ký tự hex thường), `chunk_size` và texttttotal_chunks. Relay tự thêm trường `sender` từ phiên đã xác thực, không bao giờ tin giá trị client tự khai, và từ chối OFFER có kích thước hoặc số chunk không nhất quán. File rỗng không có chunk nào và đi thẳng từ ACCEPT sang FIN.

Đăng ký điểm cuối UDP. Trước khi dùng UDP, client phải có ticket do server cấp qua `FILE_SESSION`. Sau đó client gửi BIND, relay trả CHALLENGE chứa số ngẫu nhiên 16 byte gắn với điểm cuối đó và có hạn 5 giây, client trả lại số này trong PROVE có xác thực, và chỉ khi đó relay mới gửi BOUND. Một bằng chứng cũ bị bắt lại không thể dùng để chuyển điểm cuối sang địa chỉ khác. Khi đăng ký lại từ điểm cuối mới, các lượt truyền đang chạy bị hủy chứ không được tiếp tục.

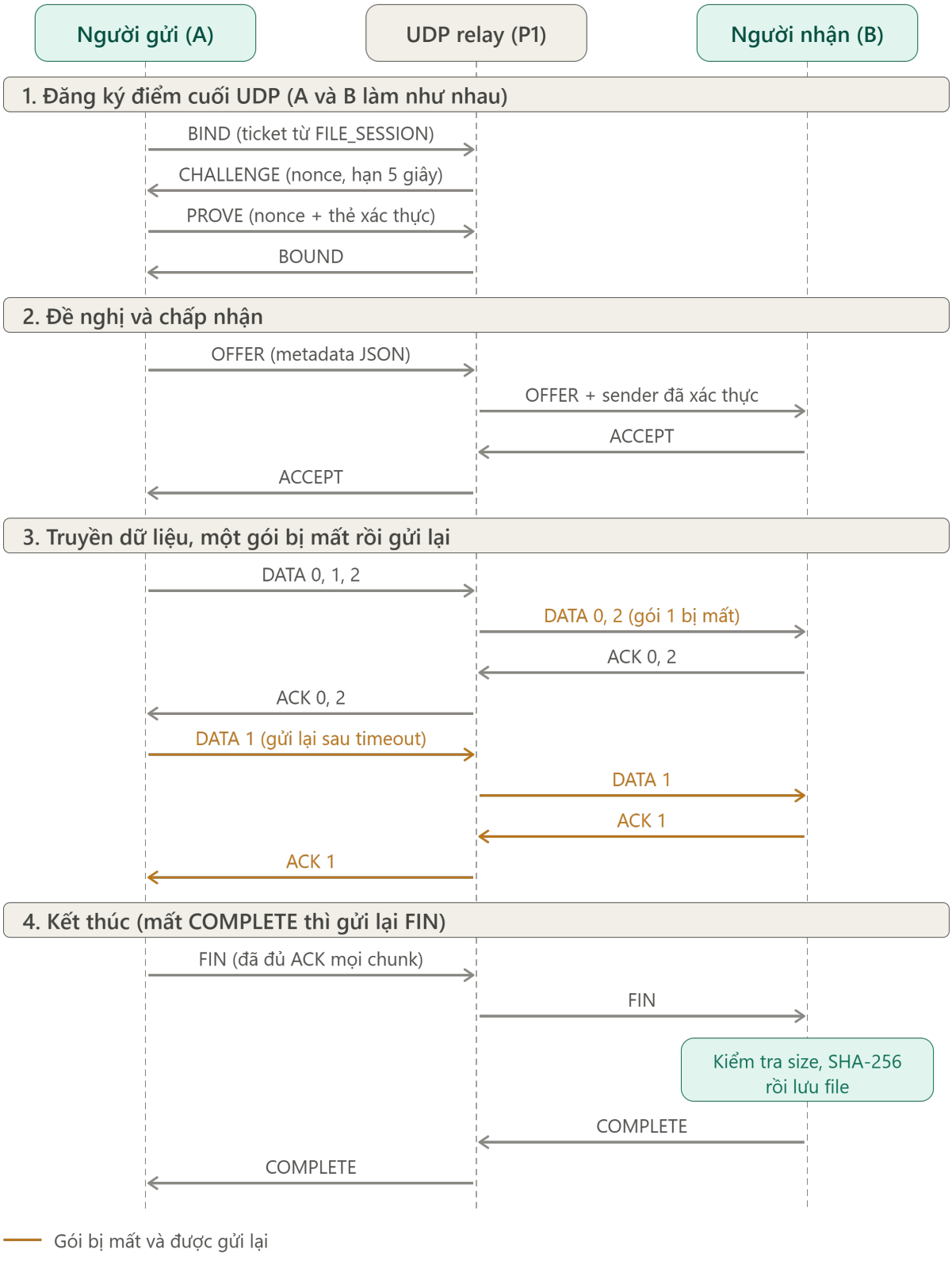
Các mã ERROR đang dùng: `INVALID_METADATA`, `TRANSFER_ID_CONFLICT`, `SELF_TRANSFER`, `RECIPIENT_OFFLINE`, `TRANSFER_LIMIT`, `PEER_DISCONECTED`, `ENDPOINT_CHANGED`, `TRANSFER_EXPIRED`, `DISK_EROR`, `INTEGRITY_OR_DISK_EROR` và `CANCLED`. Đối với thông điệp CANCEL có thể mang 2 mã `CLIENT_CLOSED` hoặc `SENDER_FAILED`, còn REJECT mặc định mang `REJECTED`. Các lỗi hết hạn hoặc hết số lần gửi lại chỉ sinh ra sự kiện `file_failed` cục bộ, không phải mã trạng thái TCP mới.

3.4. Máy trạng thái

Mỗi lượt truyền file có hai máy trạng thái độc lập, một ở người gửi và một ở người nhận:

- Người gửi: Offering → Sending (nhận ACCEPT) → Verifying (đã được ACK mọi chunk và gửi FIN) → Complete (nhận COMPLETE). Từ chối, lỗi, hết số lần gửi lại, hết hạn hoặc hủy đều dẫn tới Failed.
- Người nhận: Offered → Receiving (chấp nhận và tạo file tạm) → Verifying (nhận FIN, đủ chunk) → Complete (khớp kích thước và SHA-256 thì đổi tên nguyên tử thành file chính thức). Mọi nhánh lỗi đều xóa file tạm.

Người gửi chỉ được coi là thành công sau khi nhận COMPLETE, không phải khi đã gửi xong hay đã được ACK hết chunk.



Hình 3: Trình tự truyền một file qua UDP relay

4. Server core

4.1. Kết nối và handshake

Server lắng nghe TCP trên cổng 5000 (mặc định, cấu hình qua `main.py`) và bọc mỗi kết nối bằng TLS từ phiên bản 1.2 trở lên. Sau khi bắt tay TLS, client gửi dòng `HELLO2` và server trả `WELCOME2`. Cả hai bước đều có hạn 5 giấy (`handshake_timeout`). Nếu dòng chào sai hoặc quá dài, server trả `ERROR` rồi đóng kết nối.

Sau handshake, mỗi thông điệp là một đối tượng JSON mã hóa UTF-8, kết thúc bằng ký tự xuống dòng (LF). Server đọc theo từng dòng với deadline tính cho cả dòng, nên một thông điệp bị chia nhỏ qua nhiều lần `recv` vẫn được ghép đúng.

4.2. Xác thực và quản lý phiên

Tài khoản được `AuthManager` lưu trong trên bộ nhớ (RAM) dưới dạng từ điển (dictionary `self.users`) có khóa bảo vệ (lock). Mật khẩu không được lưu dạng thô mà được băm bằng PBKDF2-HMAC-SHA256 với salt ngẫu nhiên. Log của server chỉ ghi tên người dùng, không ghi mật khẩu.

`REGISTER` chỉ tạo tài khoản, không tự đăng nhập. Khi `LOGIN` thành công, `SessionManager` ghi lại tên người dùng, địa chỉ IP, cổng TCP của client và trạng thái `online`. Server xử lý đăng nhập trùng theo hai trường hợp: nếu kết nối này đã đăng nhập thì trả `ALREADY_AUTHENTICATED`, nếu tài khoản đang online ở kết nối khác thì trả `ALREADY_LOGGED_IN`. Trường hợp hai kết nối đăng nhập cùng lúc được xử lý bằng việc `add_session` trả `False` cho kết nối đến sau. Khi một kết nối kết thúc, phiên bị gỡ nên người dùng đăng nhập lại được ngay.

4.3. Xử lý đồng thời (Concurrency)

Server dùng mô hình mỗi kết nối một luồng (thread-per-connection) với `threading.Thread`, không dùng thread pool hay asyncio. Số kết nối tối đa là 20 (`max_clients`), được kiểm soát bằng BoundedSemaphore: khi đã đủ 20, kết nối mới bị ghi log cảnh báo và đóng ngay thay vì phải chờ. Hàng đợi chấp nhận kết nối (`backlog`) là 5.

Vì nhiều luồng cùng truy cập dữ liệu dùng chung, mỗi cấu trúc có lock riêng:

Dữ liệu dùng chung	Cơ chế bảo vệ
Tài khoản người dùng	`AuthManager._lock`
Danh sách phiên online	`SessionManager._lock`
Bảng định tuyến (socket, khóa mã hóa từng người)	`MessageRouter.lock`
Ghi dữ liệu lên socket của từng người	Send lock riêng theo từng người dùng
Danh sách socket và thread của server	`SocketServer.clients_lock`

Send lock theo từng người dùng bảo đảm hai luồng không ghi xen kẽ vào cùng một socket khi nhiều người cùng gửi tin cho một người nhận. Kiểm thử tải với 10 và 20 client đăng nhập đồng thời cho kết quả đạt, trong đó 20 client đúng bằng giới hạn `max_clients` (chi tiết ở mục 8).

4.4. Định tuyến tin nhắn

Mỗi yêu cầu được giải mã và phân loại bởi RequestDispatcher. Các action và phản hồi như sau:

Action	Phản hồi thành công	Phản hồi lỗi
`REGISTER`	`REGISTER_SUCCESS`	`REGISTER_FAILED`
`LOGIN`	`LOGIN_SUCCESS`	`LOGIN_FAILED`, `ALREADY_LOGGED_IN`, `ALREADY_AUTHENTICATED`
`UNICAST`	`UNICAST_SUCCESS`	`UNICAST_FAILED`
`BROADCAST`	`LIST_ONLINE_SUCCESS`	—
`FILE_SESSION`	`FILE_SESSION_READY`	`FILE_SESSION_FAILED`
`PING`	`PONG`	—

Các lỗi chung: `INVALID_REQUEST` (JSON sai hoặc action không hỗ trợ, server vẫn giữ kết nối), `AUTH_REQUIRED` (chưa đăng nhập mà gửi yêu cầu cần xác thực; riêng `PING` dùng được cả khi chưa đăng nhập) và `SECURITY_FAILED` (giải mã chat thất bại).

`UNICAST` kiểm tra người nhận và nội dung là chuỗi không rỗng, rồi chuyển đến đúng người nhận. `BROADCAST` gửi cho mọi người đang online trừ người gửi. `LIST_ONLINE` trả danh sách tên đã sắp xếp, bao gồm cả người hỏi. Với chat, server giải mã tin bằng khóa của người gửi rồi mã hóa lại bằng khóa riêng của từng người nhận (xem mục 6). Cần lưu ý `UNICAST_SUCCESS` xác nhận server đã gửi xong dữ liệu, không có nghĩa người nhận đã đọc tin.

Với `FILE_SESSION`, server yêu cầu UDP relay (do P1 cung cấp) cấp phiên cho người dùng và

trả ticket cùng cổng UDP. Ticket được lưu lại trong phiên TCP nên yêu cầu lặp lại không cấp ticket mới. Server cũng trả `PONG` cho `PING` để giữ kết nối không bị đóng khi đang truyền file lâu.

4.5. Xử lý lỗi và độ bền (Resillience)

Server dùng hai mức timeout: 5 giây cho handshake và 60 giây cho mỗi dòng yêu cầu khi kết nối rảnh (`idle_timeout`). Mỗi loại sự cố có cách xử lý riêng:

Sự cố	**Cách xử lý**
JSON sai hoặc action không hỗ trợ	Trả `INVALID_REQUEST`, tiếp tục phục vụ
Dòng yêu cầu quá dài	Trả `INVALID_REQUEST` rồi ngắt (trong handshake: `ERROR`)
Hết timeout hoặc client ngắt	Ghi log, thoát vòng lặp, chạy dọn dẹp
`ConnectionResetError`	Ghi cảnh báo, chạy dọn dẹp
Lỗi socket khác (`BrokenPipeError`, `ssl.SSLError`, ...)	Ghi log chi tiết, chạy dọn dẹp
Giải mã chat thất bại	Trả `SECURITY_FAILED` rồi ngắt

Dù kết thúc vì lý do nào, khối `finally` của luồng xử lý luôn thực hiện cùng một chuỗi dọn dẹp: thu hồi ticket ở UDP relay, gỡ khỏi bảng định tuyến, gỡ phiên, đóng context mã hóa, đóng socket và trả lại một chỗ cho semaphore. Nhờ vậy lỗi của một client không ảnh hưởng đến các client khác. Phía relay, mọi lượt truyền file đang dở của người đó bị đánh dấu thất bại (`PEER_DISCONNECTED`) và bên còn lại được thông báo.

Khi dừng server, `stop()` đặt cờ dừng, đóng các socket và đợi từng luồng kết thúc (tối đa 5 giây mỗi luồng), sau đó đóng relay.

Trong quá trình kiểm thử tải 10 client, server từng gặp lỗi `BlockingIOError` (WinError 10035) khi đọc dữ liệu TLS ở chế độ không chặn, làm một client bị ngắt khi các client khác đang broadcast. Nhóm đã khắc phục bằng cách cho luồng đọc và luồng ghi dùng chung một lock, chỉ gọi `recv` không chặn trong lock và chờ socket sẵn sàng ở ngoài lock theo deadline ban đầu của cả dòng. Sau khi sửa, kiểm thử 10 và 20 client đều đạt.

5. Truyền file UDP tin cậy

5.1. Chia chunk và tái lắp ghép

File được chia thành các chunk, mỗi chunk mang tối đa 996 byte dữ liệu gốc. Mỗi gói DATA mang số thứ tự chunk, nên người nhận ghi dữ liệu trực tiếp vào đúng vị trí trong file (số thứ tự nhân với `chunk_size`), kể cả khi gói đến sai thứ tự. Trạng thái các chunk đã nhận được theo dõi bằng một bitmap có kích thước giới hạn (tối đa 256 KiB mỗi file). Chunk cuối chứa đúng phần dữ liệu còn lại, còn file rỗng không có gói DATA nào và đi thẳng từ ACCEPT sang FIN.

5.2. Gửi lại có chọn lọc (selective repeat)

Người nhận gửi ACK riêng cho từng chunk đã ghi xong. Người gửi giữ một cửa sổ các chunk đang chờ xác nhận, mỗi chunk có đồng hồ riêng, và chỉ gửi lại những chunk đã hết thời gian chờ chứ không gửi lại cả cửa sổ. Nếu nhận một chunk trùng lặp, người nhận chỉ gửi lại ACK mà không ghi lần hai. Người gửi chỉ giữ phần cửa sổ đang chờ trong bộ nhớ, và việc đọc file, tính hash đều dùng các lần đọc có giới hạn dung lượng. Khi bật mã hóa, mỗi lần gửi lại dùng một nonce mới (mục 6).

5.3. Kiểm soát cửa số và tốc độ

Cửa sổ ban đầu là 4 chunk và tăng dần theo các ACK sạch tới tối đa 32 chunk. Khi hết thời gian chờ, cửa sổ giảm một nửa và đồng hồ gửi lại được kéo dài (backoff). ACK của chunk đã gửi lại không được dùng để đo RTT, vì không biết ACK đó trả lời lần gửi nào. Mọi người gửi chạy đồng thời trên một client dùng chung một bộ pacing, giới hạn mặc định không quá một gói mỗi 2 ms để tránh dồn gói. Đây là một thiết kế AIMD nhỏ giống TCP, không phải BBR hay CUBIC, và nhóm không tuyên bố đáp ứng hai thuật toán đó.

Trong lần chạy CI đầu trên Windows, bộ pacing bị ảnh hưởng bởi độ phân giải thời gian chờ thô của hệ thống. Nhóm đã đổi sang các lần ngủ ngắn theo đồng hồ đơn điệu (monotonic) và vẫn kiểm tra tín hiệu dừng định kỳ.

5.4. Thông điệp điều khiển cũng có thể mất

Nếu một thông điệp điều khiển bị mất thì file có thể đã nhận đủ mà người gửi vẫn chờ mãi. Vì vậy các cặp `BIND/CHALLENGE/PROVE/BOUND`, `OFFER/ACCEPT` và `FIN/COMPLETE` đều có cơ chế gửi lại với thời gian chờ tăng dần: bắt đầu từ 1 giây, tối đa 4 giây, nhiều nhất 8 lần gửi lại. Một yêu cầu bị gửi lặp luôn nhận cùng một câu trả lời, không cấp phát lại file. Relay và client giữ lại một số kết quả cuối (receipt) có giới hạn trong 60 giây, nên khi người gửi lặp `FIN` vì mất `COMPLETE`, hệ thống trả lại đúng kết quả đã xác minh và không ghi file lần hai.

5.5. Xác minh và công bố file

Dữ liệu nhận được ghi vào một thư mục tạm riêng, duy nhất cho từng lượt, bên trong thư mục tải về mà người nhận chọn; người nhận không bao giờ dùng đường dẫn do người gửi cung cấp. Khi nhận FIN và đủ chunk, người nhận kiểm tra cả kích thước và SHA-256 của toàn bộ file, sau đó mới đổi tên nguyên tử thành file chính thức và gửi COMPLETE. File đã tồn tại không bị ghi đè, nên nhiều file trùng tên vẫn nằm trong các thư mục riêng. Việc tính hash chạy trên các worker có giới hạn số lượng, không chặn vòng nhận UDP. Nếu kích thước hoặc hash không khớp, lượt truyền thất bại, không có file nào được công bố và file tạm bị xóa.

Cần phân biệt hai việc: SHA-256 phát hiện file bị thiếu hoặc hỏng khi ghép lại, còn việc chống sửa đổi trên đường truyền do lớp xác thực gói tin (mục 6) đảm nhiệm.

5.6. Hủy, hết hạn và dọn dẹp

Mọi lượt truyền đều có hạn chót. Khi người nhận từ chối, hết số lần gửi lại, hết hạn, gặp lỗi socket hoặc bị hủy, lượt truyền thất bại, file tạm bị xóa và các lượt truyền khác vẫn chạy bình thường. CANCEL chỉ là cố gắng tối đa (best effort); nếu gói CANCEL cũng mất, việc hết hạn do không hoạt động (60 giây) bảo đảm tài nguyên vẫn được giải phóng. Nếu một bên ngắt kết nối, relay đánh dấu các lượt đang chạy là thất bại và báo cho bên còn lại.

5.7. Cô lập giữa các đường truyền và giới hạn tài nguyên

Mỗi lượt truyền có mã, không gian số thứ tự, cửa sổ và file riêng, nên một lượt bị kẹt không chặn lượt khác hay chat. Các giới hạn mặc định:

Tài nguyên	Giới hạn
Kích thước mỗi file	256 MiB
Số lượt truyền mỗi client	8
Số lượt truyền đang chạy trên relay	64
Số phiên đăng ký trên relay	20
Thời gian không hoạt động của một lượt	60 giây
Thời gian gửi tối đa của một lượt (người gọi đặt)	300 giây (mặc định)
Kích thước datagram tối đa	1.084 byte

5.8. Kết quả kiểm chứng và giới hạn

Các kiểm thử cho phần này gồm mất DATA và ACK, gói trùng và gói đảo thứ tự, mất từng loại gói điều khiển (OFFER, ACCEPT, FIN, COMPLETE...), hash sai không được công bố, và một lượt truyền bị kẹt không chặn lượt khác (chi tiết ở mục 8). File 32 MiB truyền đạt cả kiểm tra hash và dọn dẹp. File 256 MiB đã qua bước chấp nhận nhưng việc truyền trọn vẹn vẫn thất bại do quá thời gian gửi 300 giây, và nhóm không đổi code production để làm test đạt (mục 9).

6. Bảo mật

6.1. Mô hình tin cậy và phạm vi

Bảo mật được thiết kế theo từng chặng (hop-by-hop): mỗi chặng truyền dữ liệu có lớp bảo vệ và khóa riêng. Server và relay là thành phần được tin cậy: chúng giải mã nội dung rồi mã hóa lại bằng khóa của chặng kế tiếp. Vì vậy đây không phải mã hóa đầu cuối (E2EE), và server vẫn đọc được nội dung chat và file đi qua nó.

Chặng	**Lớp bảo vệ**
Trình duyệt ↔ bridge	HTTPS; không dùng HTTP
Bridge ↔ server (TCP)	TLS từ 1.2, xác minh CA và hostname; REGISTER, LOGIN và việc cấp khóa, ticket nằm trong TLS; nội dung chat được mã hóa thêm bằng AES-256-GCM
Client ↔ relay (UDP)	AES-256-GCM cho từng gói, kèm thẻ HMAC xác thực

6.2. TLS và HTTPS

Lệnh `python -m security.tls` sinh một CA thử, chứng thư cho `localhost` và 127.0.0.1 cùng khóa riêng tại `runtime/tls/`. Chứng thư hết hạn sau 30 ngày và lệnh từ chối ghi đè file đã có. Khóa riêng của CA chỉ dùng lúc tạo chứng thư rồi bị bỏ đi, không được lưu lại.

Người dùng tin cậy CA do chính mình vừa tạo để trình duyệt xác minh HTTPS. Client xác minh CA và hostname nên chứng thư thiếu, sai CA hoặc sai hostname đều làm kết nối thất bại, và hệ thống không bao giờ tự hạ xuống plaintext. Khóa riêng, khóa phiên và ticket không được đưa vào Git.

6.3. Mã hóa payload bằng AES-256-GCM

Trường `message` trên kênh TCP có dạng `{v:2, sid:<UUID phiên>, data:base64(nonce 12 byte || ciphertext || tag 16 byte)}`. Văn bản gốc tối đa 4.000 ký tự, và khung TCP bảo mật tối đa 32.768 byte gồm ký tự xuống dòng. Server giải mã tin trước khi định tuyến, sau đó mã hóa lại bằng khóa riêng của từng người nhận, kể cả với broadcast.

Mỗi gói UDP là `header 44 byte || nonce 12 byte || dữ liệu mã hóa (0 đến 996 byte)`

`|| thẻ GCM 16 byte || thẻ HMAC 16 byte`, tức tối đa 1.084 byte. Thẻ HMAC tính trên header và phần mã hóa bằng khóa riêng của ticket; khóa này không dùng làm khóa AES. Quy trình đăng ký điểm cuối (BIND, CHALLENGE, PROVE, BOUND) cũng được bảo vệ bằng AES-GCM và HMAC. Relay giải mã gói trước khi kiểm tra và định tuyến, rồi mã hóa lại bằng khóa của người nhận đích. Bộ giải mã phiên bản 2 từ chối gói phiên bản 1 dù biết khóa HMAC, và không tự đoán phiên bản hay hạ cấp.

Kích thước và hash của file được tính trên dữ liệu gốc (plaintext). Dữ liệu chỉ được ghi xuống đĩa sau khi giải mã thành công.

6.4. Quản lý khóa theo phiên

Khi `LOGIN` thành công, phản hồi (đi trong TLS) chứa mã phiên (UUID) và hai khóa AES 32 byte cho hai chiều, `c2s` (client gửi server) và s2c (server gửi client). Server tạo khóa mới cho mỗi phiên và gửi phản hồi trước khi công bố người dùng là online. Phần khóa này chỉ do server giữ trong bộ nhớ và không bao giờ được trả ra trình duyệt.

Với file, `FILE_SESSION` cấp thêm ticket gồm tên người dùng, mã phiên, khóa HMAC riêng và hai khóa AES dành cho file. Hai khóa file khác hoàn toàn hai khóa chat. Yêu cầu `FILE_SESSION` lặp lại trong cùng phiên trả về ticket cũ và không đặt lại bộ đếm nonce. Khi ngắt kết nối, server thu hồi route, context mã hóa và ticket ở relay. Mỗi lần LOGIN mới cấp khóa và UUID mới, nên không thể tái tạo hay đặt lại context bằng ticket cũ. Khóa và ticket không bao giờ được ghi ra file.

6.5. Nonce, dữ liệu xác thực kèm theo (AAD) và chống replay

Mỗi chiều của mỗi context giữ một bộ đếm nonce 96 bit, bắt đầu từ 0 và được bảo vệ bằng lock; bộ đếm không bao giờ được đặt lại hay sao chép. Nhờ đó một khóa không bao giờ dùng lại cùng một nonce.

Dữ liệu xác thực kèm theo (AAD) buộc mỗi bản mã vào đúng ngữ cảnh của nó: chuỗi tiền tố `Connect-v2/`, mục đích (chat hay file), mã phiên và chiều truyền. Với chat, AAD còn gồm loại tin, người gửi và người nhận (với broadcast, người nhận là rỗng); với file, AAD gồm toàn bộ header 44 byte. Vì vậy một gói bị chuyển sang phiên khác, đổi chiều hoặc bị sửa header sẽ không giải mã được.

Chống replay dùng một cửa sổ bitmap 4.096 bộ đếm cho mỗi chiều. Cửa sổ chỉ cập nhật sau khi GCM xác thực thành công. Gói đến muộn hoặc đảo thứ tự trong cửa sổ vẫn được nhận, còn gói trùng nonce hoặc quá cũ bị bỏ. Khi người gửi truyền lại một chunk (mục 5.2), hệ thống giữ nguyên mã lượt truyền và số thứ tự nhưng mã hóa lại với nonce mới.

6.6. Xử lý lỗi bảo mật và tính đúng đắn của kết quả

Nếu giải mã chat thất bại do sai khóa, sai thẻ hay sai AAD, server trả `SECURITY_FAILED` rồi đóng phiên, không định tuyến tin và không ghi nội dung vào log. Chat không tự gửi lại; nếu thiếu xác nhận thì kết quả được coi là chưa xác định.

Giao diện chỉ hiển thị "Đã gửi" sau khi nhận ACK và "Hoàn tất" sau khi nhận COMPLETE hợp lệ (mục 5.5). Lớp mã hóa không chứng minh người nhận đã đọc tin, nên giao diện không diễn giải ACK thành "đã đọc".

6.7. Phạm vi kiểm chứng và giới hạn

Bộ kiểm thử bảo mật kiểm tra các trường hợp sai khóa, sai thẻ, sai AAD, nonce, replay và gửi lại, chứng thư và kết nối lại (số liệu chi tiết ở mục 8). Bản demo `/local-demo/` chạy ở chế độ mô phỏng, không chứa khóa hay kết nối thật, và không được dùng để kết luận về mã hóa.

Các giới hạn cần ghi nhận:

- Không phải E2EE; server và relay đọc được nội dung.
- Chứng thư và CA là bản thử, tọa cục bộ và hết hạn sau 30 ngày; chưa có chứng thư công khai.
- Chưa kiểm thử trên mạng LAN nhiều máy và chưa hardening cho môi trường công khai.
- Chưa có khóa nhóm, cơ sở dữ liệu hay khả năng nối lại phiên (resume).

7. Giao diện Web

7.1. Kiến trúc: trình duyệt, bridge và backend

Trình duyệt không nói chuyện trực tiếp với server hay relay. Giữa chứng có một bridge viết bằng Python (`bridge.py`), phục vụ trang web qua HTTPS (mặc định cổng 8766) và làm cầu nối tới backend:

- Chat và danh sách online: bridge duy trì kết nối TLS tới server core và chuyển sự kiện cho trình duyệt.
- File: tệp người dùng chọn được đưa vào vùng lưu tạm (staging) có giới hạn và gắn với đúng phiên, rồi một worker dùng `FileClient` gửi qua từng UDP relay. Phía nhận cũng đi theo đường ngược lại, và người dùng tải file về từ bridge.

Khóa phiên và ticket chỉ nằm ở phía bridge. Chúng không bao giờ được chuyển cho JavaScript trong trình duyệt. Phần giao diện (renderer và reducer) được tách khỏi lớp kết nối với backend, đúng yêu cầu của đề về việc tách sử lý mạng, xử lý dữ liệu và giao diện.

7.2. Luồng sử dụng

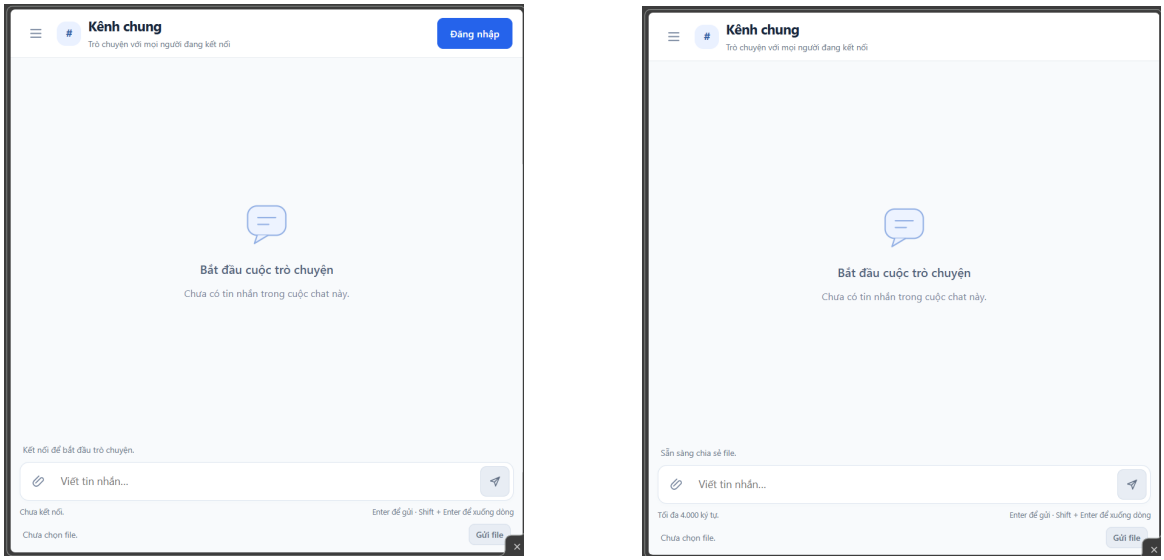
Người dùng mở `https://127.0.0.1:8766/`, đăng nhập bằng tài khoản đã có (giao diện chỉ có chức năng đăng nhập, tài khoản thử được tạo qua `REGISTER` trên kênh TLS), rồi thực hiện các thao tác sau:

(1) Danh sách người dùng: Thanh bên liệt kê các cuộc trò chuyện và mục Người đang hoạt đồng, chỉ lọc những người đang online. NGười dùng tìm người theo tên bằng nút kính lúp, tìm không phân biệt hoa thường và dấu.

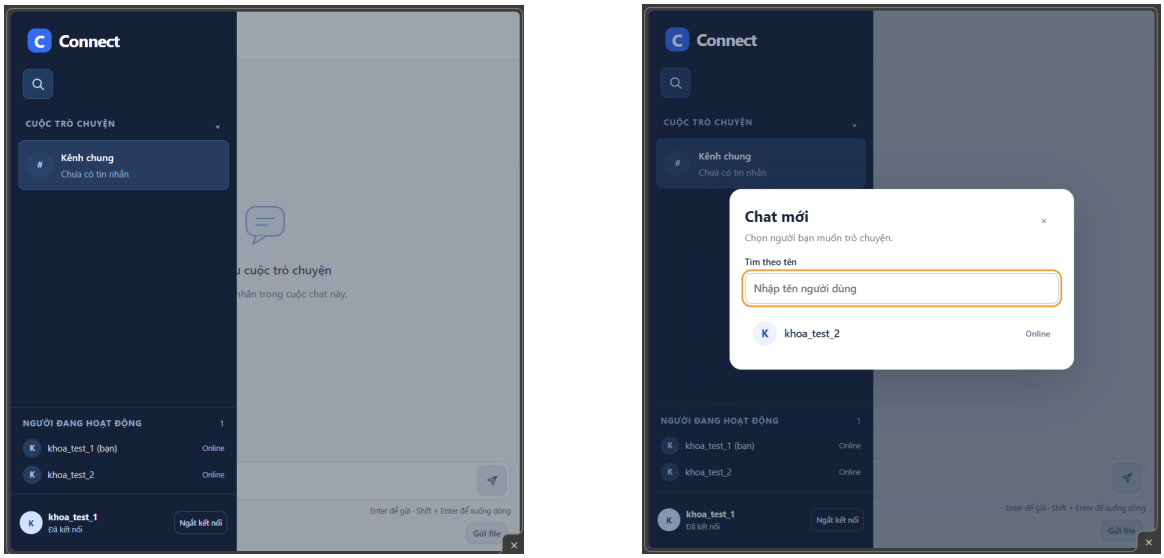
(2) Chat chung và chat riêng. Người dùng chọn "Kênh chung" (trò chuyện với mọi người đang kết nối) hoặc một người cụ thể. Tiêu đề luôn ghi loại chat và người nhận để tránh gửi nhầm. Bản nháp và lịch sử được giữ riêng cho từng cuộc trò chuyện.

(3) Gửi và nhận file. Người gửi chọn file trong cuộc chat riêng. Người nhận thấy thẻ file và chọn Chấp nhận hoặc Từ chối. Thẻ hiển thị trạng thái và thanh tiến trình trong lúc truyền, sau đó người nhận tải file về và đối chiếu SHA-256. Chat vẫn dùng được trong lúc file đang truyền.

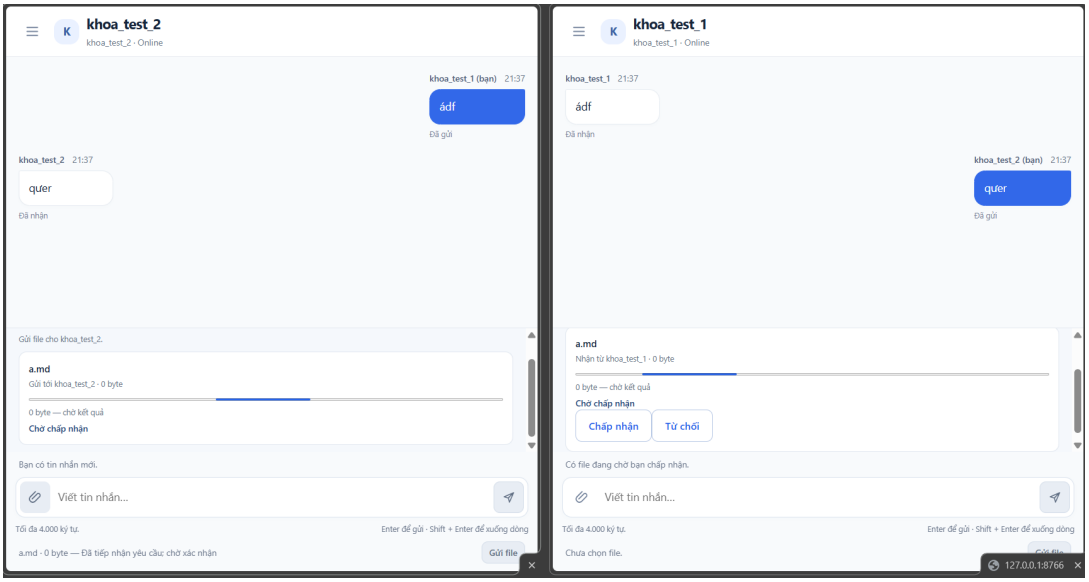
(4) Ngắt và kết nối lại. Khi người dùng ngắt kết nối, người còn lại thấy họ offline nhưng vẫn giữ nguyên cuộc trò chuyện và bản nháp; đăng nhập lại là dùng được ngay.



Hình 4: Giao diện màn hình đăng nhập và màn hình chính



Hình 5: Thanh bên và bảng tìm kiếm



Hình 6: Chat riêng với thẻ file đang truyền và thanh tiến trình

7.3. Khả nawg truy cập và thích ứng thiết bị

Trên máy tính, giao diện chia hai cột (thanh bên và khung chat), ô soạn tin luôn nằm trong vùng nhìn thấy và tự tăng chiều cao tới một giới hạn rồi cuộn bên trong. Trên điện thoại, danh sách cuộc trò chuyện thu thành ngăn kéo (drawer), có quản lý focus bằng bàn phím và đóng bằng phím Escape. Các thành phần tương tác có nhãn, trạng thái bằng chữ và focus rõ ràng. Danh sách thẻ file thu gọn được bằng bàn phím, và khi thu gọn vẫn hiển thị số lượt truyền đang chạy của cuộc trò chuyện đó.

7.4. Giới hạn

- Phạm vi kiểm chứng là localhost trên Windows, chưa kiểm thử qua nhiều máy trong LAN.
- Kích thước file lớn nhất đã kiểm thử qua trình duyệt là 8MiB.
- Chưa có hộp thư gửi offline.

8. Kiểm thử hệ thống

8.1. Công cụ và quy trình

Nhóm dùng `pytest` cho kiểm thử Python và Ruff cho lint, import và định dạng mã. Phần JavaScript của giao diện được kiểm thử bằng trình duyệt Chrome thật, vì Ruff không kiểm tra JavaScript. Môi trường chạy là Windows với Python 3.14, Node 24+ và Chrome/Edge.

Toàn bộ kiểm thử được gom vào một bộ chạy tập trung `p3.py check`, cho phép chọn từng nhóm (suite). Bộ chạy này có các cơ chế bảo đảm kết quả đáng tin cậy:

- PASS chỉ khi nhóm được chọn thật sự chạy và hoàn tất.
- Mỗi lượt chạy có UUID riêng và lưu báo cáo JSON, log, JUnit và ảnh chụp trình duyệt, dữ liệu nhạy cảm (khóa, ticket) được loại khỏi báo cáo.
- Có thời hạn tổng cho mỗi lượt, và Windows Job Object dọn toàn bộ cây tiến trình do lượt chạy tạo ra.
- Kiểm thử dùng cổng ngẫu nhiên do hệ điều hành cấp, không dừng tiến trình nào có sẵn.

Quy trình CI trên GitHub Actions chạy trên Windows với Python 3.14.

8.2. Các nhóm kiểm thử

Nhóm	Nội dung
`independent`	Kiểm tra Python độc lập với backend, Ruff, định dạng, kiểm tra JavaScript
`ui`	Kiểm thử giao diện trên Chrome ở bốn kích thước màn hình, dùng adapter giả lập
`integrated`	Server, UDP relay, bridge và giao diện chạy cùng một checkout, kèm hai phiên Chrome thật qua TLS/HTTPS
`security`	Sai khóa, sai thẻ, sai AAD, nonce, replay, gửi lại, chứng thư và thu hồi phiên
`p1`, `p2`	Hồi quy cho từng thành phần ở bản đã ghim, chạy riêng, không thay cho nghiệm thu tích hợp
`load` (tùy chọn)	10 và 20 client đồng thời, cộng thêm file và chat với 4 client
`large` (tùy chọn)	File 32 MiB và 256 MiB

Hai nhóm tùy chọn không chạy mặc định vì tốn thời gian và tài nguyên.

8.3. Kết quả

Các kết quả dưới đây ghi nhận ngày 14/11/2026 trên Windows, Python 3.14.4, dùng tải khoản và dữ liệu thử.

Nhóm	Nội dung
`independent`	Kiểm tra Python độc lập với backend, Ruff, định dạng, kiểm tra JavaScript
`ui`	Kiểm thử giao diện trên Chrome ở bốn kích thước màn hình, dùng adapter giả lập
`integrated`	Server, UDP relay, bridge và giao diện chạy cùng một checkout, kèm hai phiên Chrome thật qua TLS/HTTPS
`security`	Sai khóa, sai thẻ, sai AAD, nonce, replay, gửi lại, chứng thư và thu hồi phiên
`p1`, `p2`	Hồi quy cho từng thành phần ở bản đã ghim, chạy riêng, không thay cho nghiệm thu tích hợp
`load` (tùy chọn)	10 và 20 client đồng thời, cộng thêm file và chat với 4 client
`large` (tùy chọn)	File 32 MiB và 256 MiB

Nhóm tích hợp bao phủ chat chung và riêng hai chiều, gửi file hai chiều kèm đối chiếu SHA-256, ACK, online/offline, kết nối lại, chat trong lúc file còn đang truyền, và ngắt kết nối giữa DATA, FIN, COMPLETE. Trong lượt nghiệm thu trình duyệt thật, các file Unicode được truyền với kích thước 8.388.608 byte, 262.147 byte, 28 byte (cùng tên, nội dung mới) và file rỗng 0 byte; hash của file tải về đều khớp. Một ca từ chối file và một ca ngắt kết nối khi người nhận đang chờ chấp nhận cũng đạt, không có trạng thái hoàn tất giả.

8.4. Hạn chế của kiểm thử

- Mọi kiểm thử chạy trên localhost, trên Windows, chưa có kết quả trên mạng LAN nhiều máy.
- Chưa kiểm thử đĩa đầy và chưa đo hiệu năng (benchmark), kết quả tải chỉ cho biết các lượt đã chạy đạt, không cho biết tần suất lỗi.
- Nhóm tải và file lớn chạy riêng trên máy phát triển, không nằm trong CI mặc định.

9. Hạn chế và phát triển

Hệ thống đã đáp ứng các chức năng chính của đề trong phạm vi localhost trên Windows. Phần này liệt kê trung thực những giới hạn đã biết, để người đọc không hiểu nhầm phạm vi của kết quả, cùng với hướng phát triển để khắc phục.

Nhóm	**Hạn chế hiện tại**	**Hướng phát triển**
Dữ liệu và phiên	Tài khoản và danh sách phiên chỉ lưu trong bộ nhớ (RAM), nên khi server khởi động lại, mọi tài khoản đều mất. Chưa có cơ sở dữ liệu.	Lưu tài khoản vào cơ sở dữ liệu bền vững; giữ nguyên cách băm mật khẩu hiện có.
Ý nghĩa của xác nhận	`UNICAST_SUCCESS` chỉ cho biết server đã gửi xong dữ liệu vào socket của người nhận, không phải người nhận đã đọc. `UNICAST_FAILED` không phân biệt người nhận offline với lỗi gửi. `BROADCAST_SUCCESS` vẫn được trả về khi không có người nhận nào. Chat không tự gửi lại, và chưa có hộp thư gửi offline hay trạng thái "đã xem".	Thêm xác nhận đã nhận ở tầng ứng dụng, phân biệt rõ các mã lỗi và bổ sung hộp thư gửi offline (outbox).
Kiểm tra tên người dùng	Server chấp nhận tập tên rộng hơn quy tắc của giao thức file (1 đến 64 byte UTF-8, không khoảng trắng ở hai đầu, không ký tự điều khiển). Tài khoản có tên không hợp lệ theo quy tắc đó có thể đăng ký được nhưng cấp phiên file sẽ trả `FILE_SESSION_FAILED`.	Áp dụng cùng một quy tắc kiểm tra tên ngay từ lúc đăng ký.

Nhóm	Hạn chế hiện tại	Hướng phát triển
Bảo mật	Không phải mã hóa đầu cuối (E2EE): server và relay đọc được nội dung. Chứng thư và CA là bản thử, tạo cục bộ và hết hạn sau 30 ngày. Chưa hardening cho môi trường công khai và chưa có khóa nhóm.	Nghiên cứu E2EE và khóa nhóm; dùng chứng thư từ CA thật khi triển khai ra ngoài localhost.
Chống dò mật khẩu	Chưa có giới hạn số lần đăng nhập hoặc đăng ký, trong khi mỗi lần băm mật khẩu tốn nhiều tài nguyên.	Thêm giới hạn tần suất (rate limit) và khóa tạm thời theo tài khoản hoặc địa chỉ.
Giao thức và yêu cầu của đề	Chat đi qua TCP, chỉ truyền file đi qua UDP. Đề có nhắc UDP streaming và các thuật toán BBR, CUBIC; cơ chế điều khiển tắc nghẽn của nhóm là một thiết kế AIMD đơn giản, không phải BBR hay CUBIC. Các điểm mơ hồ này chưa được giảng viên làm rõ.	Làm rõ yêu cầu với giảng viên; nếu cần, bổ sung chat qua UDP hoặc điều khiển tắc nghẽn nâng cao.
Truyền file lớn	File 256 MiB truyền thất bại do quá thời gian gửi 300 giây. File tối đa 256 MiB. Khi đăng ký lại điểm cuối UDP, các lượt truyền đang chạy bị hủy chứ không được tiếp tục; chưa có chức năng nối lại (resume). File lớn nhất đã kiểm thử qua trình duyệt là 8 MiB.	Điều tra và xử lý nguyên nhân quá thời gian ở file lớn; thêm cơ chế resume.
Phạm vi kiểm thử	Mọi kiểm thử chạy trên localhost, trên Windows. Chưa kiểm thử LAN nhiều máy, đĩa đầy hay hiệu năng.	Chạy nghiệm thu trên mạng LAN hoặc VPN nhiều máy; bổ sung kiểm thử đĩa đầy và benchmark.

Vận hành server	Điểm khởi động chưa xử lý tín hiệu dừng, nên việc tắt server chưa được dọn dẹp chủ động.	Bắt tín hiệu dừng và gọi quy trình dừng sạch của server.

Những hạn chế trên đều đã được ghi nhận trong quá trình kiểm thử và không ảnh hưởng đến các kết quả đã trình bày ở mục 8. Nhóm không tuyên bố hệ thống sẵn sàng cho môi trường công khai.

10. Kết luận

Nhóm đã xây dựng ứng dụng Connect với đầy đủ các chức năng chính của đề: đăng nhập và quản lý phiên, nhiều client đồng thời, chat chung và riêng, truyền file tin cậy qua UDP có kiểm tra SHA-256, xử lý lỗi và ngắt kết nối, mã hóa dữ liệu bằng TLS và AES-256-GCM, và giao diện Web hiển thị tin nhắn, người dùng online và tiến trình truyền file. Hệ thống được kiểm thử tự động với 188 test tích hợp, 23 test bảo mật, kiểm thử trình duyệt thật và các lượt chạy tải 10 và 20 client đồng thời, tất cả đều đạt.

Quá trình làm việc cho thấy giá trị của kiểm thử tích hợp: hai lỗi khó thấy, lỗi đọc TLS khi nhiều client broadcast và lỗi FIN lặp chặn vòng nhận UDP, đều chỉ lộ ra khi các thành phần chạy cùng nhau trong tải đồng thời, và được xử lý kèm test hồi quy. Việc tách rõ xác nhận "đã gửi", "hoàn tất" với trạng thái "chưa xác định kết quả" cũng giúp giao diện không báo thành công khi chưa có bằng chứng.

Hệ thống vẫn còn những giới hạn đã nêu ở mục 9: không phải mã hóa đầu cuối, dữ liệu tài khoản chỉ nằm trong bộ nhớ và mới được kiểm thử trên localhost. Một số điểm mơ hồ trong đề (giao thức của chat, thuật toán điều khiển tắc nghẽn) cần được giảng viên làm rõ. Các hướng phát triển tiếp theo gồm lưu trữ bền vững, kiểm thử trên mạng LAN, xử lý file lớn và cải thiện bảo mật.